

CS 650 – Full Stack Application Develop

**3-2 Activity: Component- Based UI Build**

Malgorzata Debska

Southern New Hampshire University

Brian Enochson

July 22, 2026

## UI Components for the Animal Shelter Adoption Platform

The animal shelter adoption platform was designed using a component-based architecture to create a clean, organized, and maintainable user interface. Rather than placing all of the application's functionality into a single file, the interface is divided into smaller, reusable components that each perform one specific task. This design approach improves readability, simplifies debugging, encourages code reuse, and makes future enhancements easier to implement. If additional features such as searching, sorting, or user authentication are added later, they can be incorporated without requiring significant changes to the existing codebase.

The first component is the **Header** component. This component appears at the top of the application and displays the animal shelter's logo and name. Since the header remains consistent across the application, separating it into its own component allows it to be reused on multiple pages if the application expands in the future. The Header component is responsible only for displaying branding information and does not contain any application logic. Directly beneath the header is the **Filter Panel** component. This component allows visitors to narrow the list of available animals by selecting different filtering options. Two filtering controls are provided: one for the type of animal (dog, cat, or rabbit) and another for the animal's availability status. When users change one of these filter options, the selected values are communicated to the parent component, which updates the displayed list of animals. Keeping the filtering functionality separate from the rest of the interface makes the code easier to maintain and allows additional filters to be added later with minimal changes. The **Animal Grid** component serves as the container for all of the animal cards displayed on the screen. After receiving data from the back-

end API, the grid component iterates through the collection of animals and generates an individual Animal Card component for each record. Organizing the cards inside a dedicated grid component provides a consistent layout while separating layout responsibilities from data presentation. Each animal is represented by an **Animal Card** component. This reusable component displays the basic information needed for visitors to browse adoptable animals quickly. Every card includes the animal's photograph, name, breed, age, and a badge indicating whether the animal is currently available for adoption. Because the same layout is used for every animal, creating a reusable card component significantly reduces duplicate code while ensuring a consistent appearance throughout the application. When an Animal Card is selected, an **Animal Details** component expands beneath the summary of information. This component provides more detailed information about the selected animal, including its medical history, temperament notes, and an Adoption Inquiry button. Although the inquiry button does not perform any action in this activity, it represents where future functionality could be implemented. Separating the detailed information into its own component prevents the Animal Card from becoming overly complex and keeps responsibilities clearly defined.

## UI Component Hierarchy

The application follows a hierarchical component structure in which each component has a clearly defined responsibility. The App component serves as the root component and manages application state. It coordinates communication between the service layer and the UI components while passing data to child components through properties (props).

## **App**

Header

FilterBar

Main Content

Loading Message or Error Message

AnimalGrid

AnimalCard (one card for each API record)

AnimalDetail (conditionally displayed when selected)

This hierarchical organization improves scalability because each component focuses on a single responsibility. Individual components can be modified, tested, or reused independently without affecting the rest of the application.

## **Service Layer**

To separate business logic from presentation logic, a dedicated service layer was created to manage communication with the back-end API. Rather than allowing UI components to make HTTP requests directly, all API interactions are centralized within the service layer. This follows software engineering best practices by improving maintainability, reducing duplicate code, and making future API changes easier to implement.

The service layer includes two methods:

- **getAnimals()** retrieves the complete list of animals available from the back-end API.

- **getAnimalById(id)** retrieves detailed information for an individual animal when additional information is needed.

Using a service layer also improves testing because API functionality can be tested independently of the user interface. If the API endpoint changes in the future, only the service layer requires modification while the UI components remain unchanged.

## **Displaying Data from the Back-End API**

After the application loads, the service layer sends a request to the back-end API to retrieve the available animal records. Once the data is returned, it is stored within the application state and passed to the Animal Grid component. The Animal Grid dynamically generates an Animal Card component for each animal received from the API using React's JSX templating engine.

Each Animal Card displays the following information retrieved from the API:

- Animal photograph
- Animal name
- Breed
- Age
- Availability badge

When users click on an Animal Card, the Animal Details component expands to display additional information, including the animal's medical history, temperament notes, and an Adoption Inquiry button. Because the information is rendered dynamically, the interface

automatically updates whenever new data is received from the back-end API. This eliminates the need to manually modify the interface whenever animals are added, removed, or updated within the system.

Using reusable components together with a centralized service layer creates a well-organized front-end architecture that is easier to maintain, extend, and test. This approach reflects industry best practices for modern full stack application development by separating presentation, business logic, and data retrieval into distinct layers while providing a responsive and user-friendly experience for visitors searching for adoptable animals.

## Screenshots



